



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment – 05

Missionaries and Cannibals Problem

Aim

To solve the Missionaries and Cannibals problem using the State Space Search algorithm.

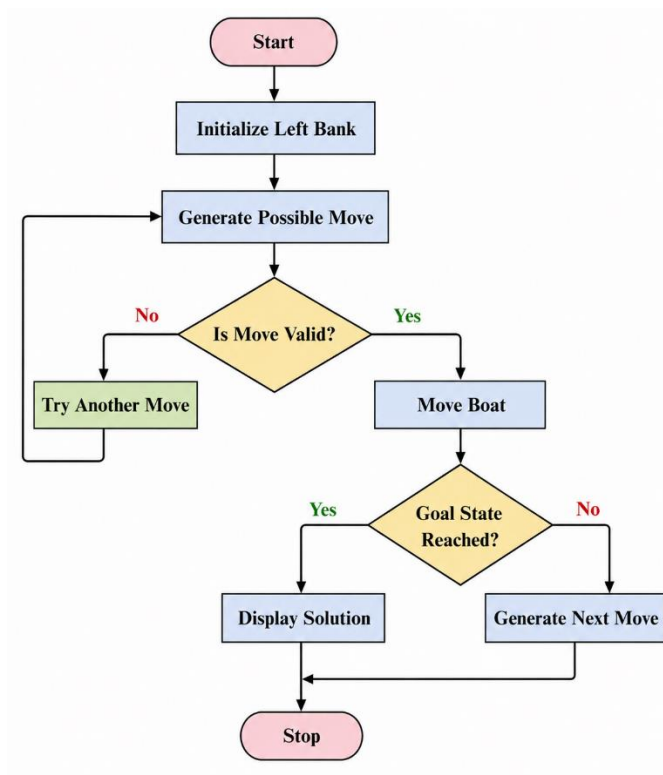
Objective

To safely transport three missionaries and three cannibals from the left river bank to the right river bank without ever allowing the cannibals to outnumber the missionaries on either bank.

Algorithm

1. Start with all missionaries and cannibals on the left bank.
2. Place the boat on the left bank.
3. Generate all valid boat moves (one or two people).
4. Check whether the new state is valid.
5. If valid, move the boat to the opposite bank.
6. Repeat the process until all missionaries and cannibals reach the right bank.
7. Display the sequence of valid states.
8. Stop.

FlowChart



Code:

```
from collections import deque
```

```
def is_valid(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if (m > 0 and m < c):
```

```
        return False
```

```
    if (3 - m > 0 and (3 - m) < (3 - c)):
```

```
        return False
```

```
    return True
```

```
def solve():
```

```
    start = (3, 3, 1)
```

```
    goal = (0, 0, 0)
```

```
queue = deque([(start, [])])
```

```
visited = set()
```

```
while queue:
```

```
    state, path = queue.popleft()
```

```
    if state == goal:
```

```
        return path + [state]
```

```
    if state in visited:
```

```
        continue
```

```
    visited.add(state)
```

```
    m, c, boat = state
```

```
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
```

```
    for dm, dc in moves:
```

```
        if boat == 1:
```

```
            new = (m-dm, c-dc, 0)
```

```
        else:
```

```
            new = (m+dm, c+dc, 1)
```

```
        if is_valid(new[0], new[1]):
```

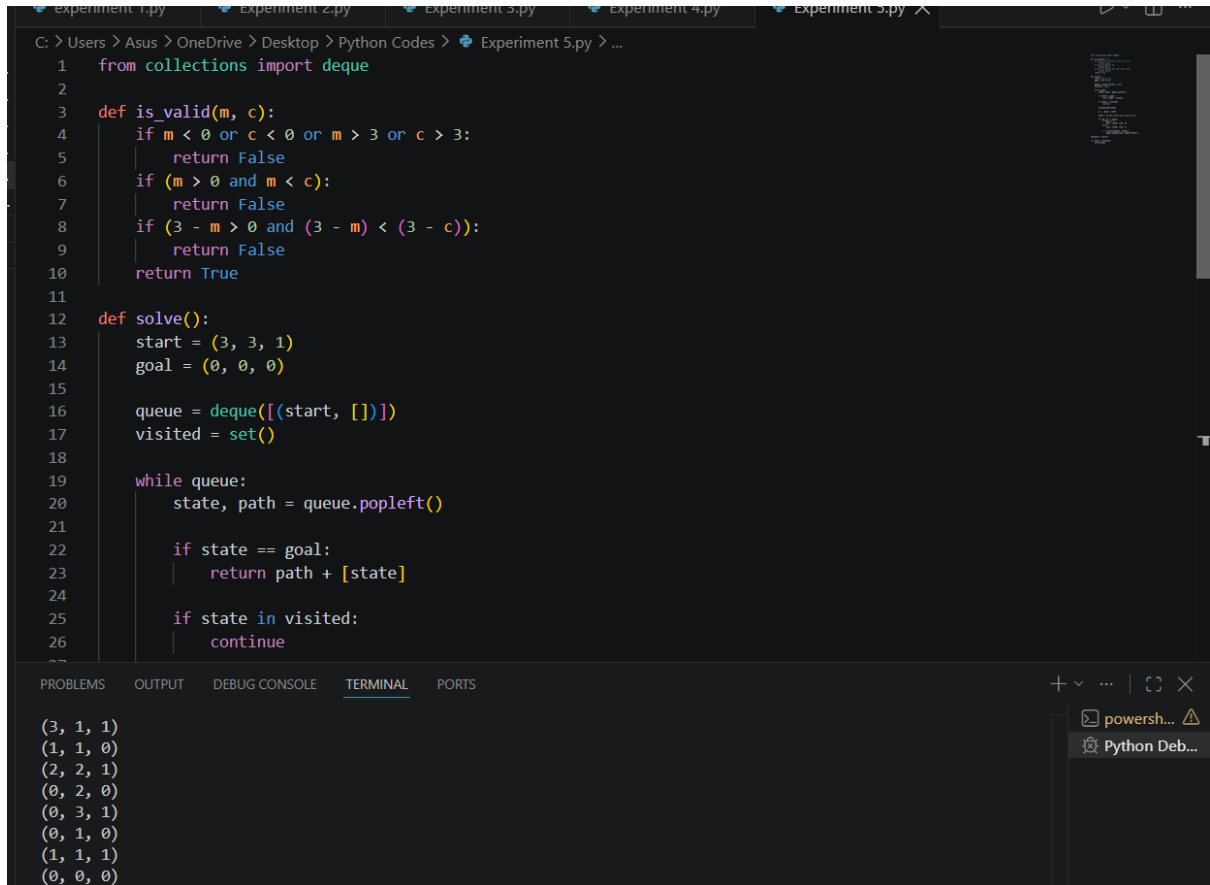
```
            queue.append((new, path+[state]))
```

```
solution = solve()
```

for step in solution:

print(step)

Output:



The screenshot shows a Python IDE with a file named 'Experiment 5.py'. The code defines a function 'is_valid(m, c)' that checks if a state is valid based on the constraints of the Missionaries and Cannibals problem. It then defines a 'solve()' function that uses a breadth-first search algorithm to find a solution. The 'solve()' function starts with an initial state (3, 3, 1) and a goal state (0, 0, 0). It uses a queue to explore states and a set to track visited states. The solution found is a list of states: (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), and (0, 0, 0).

```
1 from collections import deque
2
3 def is_valid(m, c):
4     if m < 0 or c < 0 or m > 3 or c > 3:
5         return False
6     if (m > 0 and m < c):
7         return False
8     if (3 - m > 0 and (3 - m) < (3 - c)):
9         return False
10    return True
11
12 def solve():
13    start = (3, 3, 1)
14    goal = (0, 0, 0)
15
16    queue = deque([(start, [])])
17    visited = set()
18
19    while queue:
20        state, path = queue.popleft()
21
22        if state == goal:
23            return path + [state]
24
25        if state in visited:
26            continue
27
28    return None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

Result

The Missionaries and Cannibals problem was solved successfully using the State Space Search algorithm.

Conclusion

The program successfully finds a valid sequence of moves to transport all missionaries and cannibals safely across the river without violating the problem constraints.